



Institut Teknologi Bandung

Direktorat Pendidikan Profesional Berkelanjutan

×

CHAPTER 6

Alur Kerja Universal Machine Learning

Anda tidak memulai dari kumpulan data. Anda memulai dari persoalan – dan pekerjaan tersulitnya justru sebelum baris kode pertama.

Durasi 3 jam (2 sesi)

Pengajar Prof. Bambang Riyanto Trilaksono

Sumber Chollet & Watson, Deep Learning with Python 3e – bab 6 (hlm. 171-189)

Tujuan Sesi

Setelah sesi ini, peserta dapat:

- 1 Merumuskan sebuah persoalan bisnis menjadi **jenis tugas machine learning** yang tepat – termasuk mengenali kapan machine learning **bukan** jawabannya.
- 2 Menyebut **dua hipotesis** yang diam-diam Anda buat setiap kali memulai proyek, dan apa artinya bila keduanya salah.
- 3 Merancang pengumpulan dan anotasi data yang **mewakili data produksi**, dan mengenali **bias pencuplikan**, **kebocoran target**, serta **pergeseran konsep**.
- 4 Memilih ukuran keberhasilan, lalu memilih **aktivasi lapis akhir**, **fungsi rugi**, dan **metrik** yang sepadan dengannya.
- 5 Menjalankan tiga tahap pengembangan model: **kalahkan tolak banding** → **besarkan sampai overfit** → **regularisasi dan setel**.
- 6 Memilih cara penyerahan – **REST API**, **di perangkat**, atau **di peramban** – beserta pengoptimalan inferensinya (pemangkasan dan kuantisasi bobot).
- 7 Menyusun **penetapan harapan pemangku kepentingan** dalam bahasa false-positive dan false-negative, bukan 'akurasi 98%'.

BUKU

[Bab 6 — teks penuh](#)

NOTEBOOK

[01_memahami_data_sebelum_model.ipynb](#)

NOTEBOOK

[02_pra_pemrosesan_umum.ipynb](#)

NOTEBOOK

[03_ekspor_dan_kuantisasi.ipynb](#)

Di dunia nyata, `keras.datasets` tidak ada

Bayangkan Anda membuka jasa konsultasi machine learning sendiri. Proyeknya mulai berdatangan – dan **tidak satu pun datang bersama kumpulan datanya**

Pencarian foto

Ketik *wedding*, dapatkan semua foto pernikahan – tanpa penandaan manual.

Spam & konten kasar

Menandai kiriman pada aplikasi obrolan yang baru tumbuh.

Perekomendasi musik

Untuk pengguna radio daring.

Fraud kartu kredit

Untuk sebuah situs niaga elektronik.

Click-through rate iklan

Memutuskan iklan mana yang disajikan ke siapa, kapan.

Kue cacat

Menandai kue janggal di ban berjalan pabrik.

Situs arkeologi

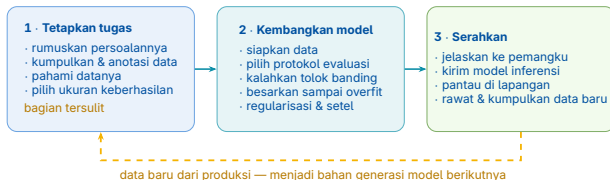
Menebak lokasi situs yang belum diketahui dari citra satelit.

...dan kasus Anda sendiri

Yang akan dibawa ke tugas kelompok nanti.

Ketujuh contoh ini dipakai berulang sepanjang bab, dan tiap tahap alur kerja diuji terhadap ketujuhnyanya. **Perhatikan bahwa dua di antaranya ternyata bukan persoalan deep learning sama sekali.**

Tiga bagian, dan yang tersulit ada di depan



Alur kerja universal – dan panah balik yang membuatnya tidak pernah benar-benar selesai.

*Pengembangan model hanyalah satu langkah dalam alur kerja machine learning, dan menurut kami bukan yang tersulit. Yang paling sulit adalah **merumuskan persoalan serta mengumpulkan, menganotasi, dan membersihkan data.***

Chollet & Watson, bab 6.2

01

Menetapkan tugas

Anda tidak bisa bekerja baik tanpa memahami konteksnya secara mendalam.

Empat pertanyaan yang harus ada di kepala

- ❶ **Apa data masukannya? Apa yang hendak diramalkan?** Anda hanya bisa belajar meramalkan sesuatu bila ada data latihnya. **Ketersediaan data biasanya jadi faktor pembatas di tahap ini.**
- ❷ **Jenis tugas machine learning apa ini?** Biner? Multikelas? Regresi skalar? Segmentasi citra? Perangkingan? Atau – mungkin saja – **machine learning bukan cara terbaik**, dan analisis statistik biasa lebih tepat.
- ❸ **Seperti apa penyelesaian yang sudah ada?** Mungkin pelanggan sudah punya algoritma buatan tangan dengan banyak **if** bersarang. Mungkin ada manusia yang sekarang mengerjakannya secara manual. Pahami sistem yang sudah berjalan.
- ❹ **Adakah kendala khusus?** Aplikasi terenkripsi ujung-ke-ujung → model harus hidup di ponsel pengguna. Kendala latensi ketat → model harus jalan di perangkat tertanam di pabrik, bukan di server jauh.

Perhatikan pertanyaan ketiga. Di banyak proyek nyata, **tolok bandingnya bukan 'acak', melainkan sistem berbasis aturan yang sudah berjalan bertahun-tahun** – dan itu jauh lebih sulit dikalahkan.

Ketujuh contoh, dipetakan ke jenis tugasnya

PROYEK	JENIS TUGASNYA	CATATAN
Pencarian foto	Klasifikasi multikelas, multilabel	—
Spam	Klasifikasi biner	Jadi tiga kelas kalau <i>konten kasar</i> dijadikan kelas tersendiri.
Perekomendasi musik	Bukan deep learning	Lebih baik ditangani faktorisasi matriks (collaborative filtering).
Fraud kartu kredit	Klasifikasi biner	—
Click-through rate	Regresi skalar	—
Kue cacat	Klasifikasi biner	Tetapi butuh deteksi objek dulu untuk memotong kuenya dari citra mentah. Catatan buku: teknik yang dikenal sebagai <i>anomaly detection</i> justu tidak cocok di sini
Situs arkeologi	Perangkingan kemiripan citra	Mengambil citra yang paling mirip situs yang sudah dikenal.

Dua pelajaran dari tabel ini: ada persoalan yang **bukan** deep learning, dan ada persoalan yang **butuh dua model bertahap**. Keduanya sering terlewat kalau langsung meloncat ke pemodelan.

Dua hipotesis yang selalu Anda buat diam - diam

Target bisa diramalkan dari masukan

Bahwa hubungan itu memang ada.

Data yang ada cukup informatif

Untuk mempelajari hubungan antara masukan dan target itu.

Sampai Anda punya model yang bekerja, keduanya **sekadar hipotesis** yang menunggu dibuktikan atau digugurkan. Menyusun contoh X dan target Y **tidak berarti X memuat cukup informasi untuk meramal Y**

Contoh yang diberikan buku: meramalkan pergerakan saham dari riwayat harganya saja **tidak mungkin berhasil**, sebab riwayat harga tidak memuat banyak informasi yang meramalkan.

Kalau setelah mencoba beberapa arsitektur yang masuk akal Anda tetap tidak bisa mengalahkan tolok banding sederhana, besar kemungkinan **jawaban atas pertanyaan Anda memang tidak ada di dalam data masukannya**. Kembali ke papan gambar.

Teknologi tidak pernah netral

- ① **Kesahihannya sendiri meragukan** – tidak jelas mengapa sifat dapat-dipercaya akan tercermin di wajah seseorang.
- ② Mengumpulkan datanya sama saja dengan **merekam bias dan prasangka orang-orang yang melabeli fotonya**.
- ③ Model yang dilatih di atasnya hanya akan **menyandikan bias yang sama ke dalam algoritma kotak hitam** – yang justru memberinya lapisan tipis keabsahan.

Di masyarakat yang sebagian besar belum melek teknologi seperti kita, "algoritma AI mengatakan orang ini tidak bisa dipercaya" anehnya terdengar lebih berbobot dan lebih objektif daripada "John Smith mengatakan orang ini tidak bisa dipercaya" – padahal yang pertama adalah hampiran terpelajar atas yang kedua.

Chollet & Watson, bab 6.1.1

Teknologi tidak pernah netral. Kalau pekerjaan Anda berdampak pada dunia, dampak itu punya arah moral: **pilihan teknis juga pilihan etis**. Selalu sengaja memilih nilai apa yang hendak didukung pekerjaan Anda.

Mengumpulkan data – bagian termahal, dan yang paling berimbal hasil

*Kalau Anda punya tambahan 50 jam untuk sebuah proyek, kemungkinan besar cara paling efektif membelanjakannya adalah **mengumpulkan lebih banyak data**, bukan mencari perbaikan pemodelan yang sedikit demi sedikit.*

Chollet & Watson, bab 6.1.2

Titik bahwa **data lebih penting daripada algoritma** paling terkenal dikemukakan makalah peneliti Google tahun 2009, "The Unreasonable Effectiveness of Data" – judulnya memecahkan buku Eugene Wigner 1960, *The Unreasonable Effectiveness of Mathematics in the Natural Sciences*. Itu ditulis **sebelum** deep learning populer, dan naiknya deep learning **justro menambah** pentingnya data.

PILIHAN ANOTASI	UNTUNGNYA	RISIKONYA
Anotasi sendiri	Kendali penuh atas mutu.	Lambat dan mahal dalam waktu.
Urun daya (mis. Mechanical Turk)	Murah dan berskala baik.	Anotasinya bisa cukup berderau
Perusahaan pelabelan khusus	Menghemat waktu dan biaya.	Melepaskan kendali.

- Apakah pelabelnya **harus ahli bidang**? Kucing vs anjing bisa siapa saja; ras anjing perlu pengetahuan khusus; CT scan patah tulang praktis menuntut gelar kedokteran.
- Kalau perlu keahlian, **bisakah orang dilatih** untuk itu? Kalau tidak, bagaimana Anda mengakses ahlinya?
- **Apakah Anda sendiri paham** bagaimana ahli itu sampai pada anotasinya? Kalau tidak, data Anda jadi kotak hitam dan rekayasa fitur manual tertutup – tidak fatal, tetapi membatasi.
- Perangkat lunak anotasi yang produktif **menghemat sangat banyak waktu**; layak diinvestasikan sejak awal proyek.

Data yang tidak mewakili – dosa besar

Contoh dari buku. Aplikasi pengenalan masakan dari foto. Model dilatih dengan foto dari jejaring sosial berbagi gambar yang populer di kalangan penggemar kuliner. Akurasi uji jauh di atas 90%.

Setelah dirilis: **salah 8 dari 10 kali**. Foto unggahan pengguna – masakan acak, restoran acak, ponsel acak – **sama sekali tidak menyerupai** foto profesional yang pencahayaannya bagus dan menggugah selera itu.

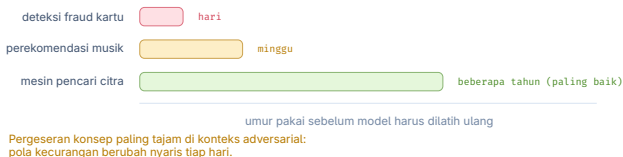
Aturannya

Bias pencuplikan adalah bentuknya yang paling licik: proses

- Bila mungkin, kumpulkan data **langsung dari lingkungan tempat model akan dipakai**.
- Model sentimen ulasan film dipakai pada ulasan IMDB baru – bukan pada ulasan restoran Yelp, bukan pada status Twitter.
- Kalau melatih di data produksi tidak mungkin, **pahami betul bedanya** dan koreksi perbedaan itu secara aktif.

pengumpulan data berinteraksi dengan hal yang hendak Anda ramalkan. Contoh sejarahnya terkenal – malam pemilu 1948, *Chicago Tribune* memasang tajuk "**DEWEY DEFEATS TRUMAN**". Paginya Truman yang menang. Redaktornya memercayai survei telepon; padahal pengguna telepon pada 1948 **bukan cuplikan acak yang mewakili pemilih** – mereka cenderung lebih kaya dan konservatif.

Pergeseran konsep – dan umur pakai model



Angka-angka ini dari buku. Perhatikan bahwa deteksi fraud – kasus yang paling adversarial – adalah yang paling cepat basi.

- ♦ **Pergeseran konsep** terjadi saat sifat data produksi berubah seiring waktu, sehingga akurasi model merosot perlahan.
- ♦ Kumpulan IMDB dikumpulkan pada 2011; model yang dilatih di atasnya akan bekerja lebih buruk pada ulasan 2020 daripada ulasan 2012 – kosakata, ungkapan, dan genre film berubah.
- ♦ Menangani pergeseran yang cepat menuntut **pengumpulan data, anotasi, dan pelatihan ulang yang terus-menerus**.

Kalimat penutupnya keras: memakai machine learning yang dilatih atas data masa lalu untuk meramal masa depan berarti **mengandaikan masa depan akan berkelakuan seperti masa lalu. Sering kali tidak.**

Memahami data Anda – daftar periksa sebelum melatih

- 1 Ada citra atau teks? **Lihat langsung** beberapa contohnya, berikut labelnya.
- 2 Ada fitur numerik? **Gambar histogramnya** untuk merasakan rentang nilai dan seberapa sering tiap nilai muncul.
- 3 Ada informasi lokasi? **Petakan**. Apakah ada pola yang muncul?
- 4 Ada sampel yang **nilainya hilang**? Itu harus ditangani saat penyiapan.
- 5 Tugas klasifikasi? **Cetak jumlah contoh tiap kelas**. Kalau tidak seimbang, ketidakseimbangan itu harus diperhitungkan.
- 6 Periksa **kebocoran target**.

Kebocoran target – adanya fitur yang memberi informasi tentang target tetapi **tidak akan tersedia di produksi**. Contoh buku: melatih model atas rekam medis untuk meramal apakah seseorang akan dirawat karena kanker, sementara rekamnya memuat fitur *"orang ini telah didiagnosis kanker"*.

Pertanyaan yang harus selalu Anda ajukan: **apakah setiap fitur dalam data saya akan tersedia dalam bentuk yang sama di produksi?**

Memilih ukuran keberhasilan

Untuk mengendalikan sesuatu, Anda harus bisa mengamatinya. Untuk berhasil dalam sebuah proyek, Anda harus lebih dulu menetapkan apa yang Anda maksud dengan berhasil.

Chollet & Watson, bab 6.1.4

BENTUK PERSOALAN	METRIK YANG LAZIM
Klasifikasi seimbang	Akurasi, dan AUC dari kurva ROC.
Kelas tidak seimbang , perangkingan, multilabel	Presisi dan recall , atau metrik yang menghitung false positive / true positive / false negative / true negative.
Bukan salah satu di atas	Tidak jarang Anda harus menetapkan metrik sendiri .

Metrik keberhasilan **menuntun semua pilihan teknis** sepanjang proyek. Ia harus selaras langsung dengan sasaran yang lebih tinggi – yaitu **keberhasilan bisnis pelanggan Anda**.

Untuk merasakan keragaman metrik dan kaitannya dengan ranah persoalan, buku menyarankan menjelajahi kompetisi di [Kaggle](#).

02

Mengembangkan model

Bagian yang paling banyak diajarkan tutorial – dan bukan yang tersulit.

Menyiapkan data: vektorisasi, normalisasi, nilai hilang

Vektorisasi. Semua masukan dan target harus berupa tensor bilangan titik-mengambang (atau, dalam kasus khusus, tensor bilangan bulat atau string). Suara, citra, teks – semuanya diubah jadi tensor lebih dulu.

Normalisasi nilai. Tidak aman menyuapkan data bernilai relatif besar (bilangan bulat berdigit banyak, jauh lebih besar dari nilai awal bobot jaringan) atau data yang **heterogen** (satu fitur di 0-1, yang lain di 100-200). Itu memicu pembaruan gradien besar yang **mencegah jaringan konvergen**

dua sifat yang diinginkan

PYTHON

```
# 1. Bernilai kecil - umumnya di rentang 0-1
# 2. Homogen - semua fitur kira-kira
#    dalam rentang yang sama

# praktik yang lebih ketat, sering
# menolong walau tidak selalu perlu:
x -= x.mean(axis=0) # rerata 0
x /= x.std(axis=0)  # simpangan baku 1
# (x = matriks 2D (samples, features))
```

NILAI HILANG PADA FITUR...	YANG DILAKUKAN
Kategorik	Aman membuat kategori baru yang berarti <i>nilainya hilang</i> . Model akan belajar sendiri apa artinya terhadap target.
Numerik	Hindari mengisi nilai sembarang seperti 0 – itu bisa menciptakan ketakbersinambungan pada ruang laten. Pakai rerata atau median fitur itu; atau latih model untuk menebak nilainya dari fitur lain.

Jebakan yang halus: kalau Anda **mengharapkan** fitur kategorik hilang di data uji tetapi jaringan dilatih tanpa satu pun nilai hilang, jaringan **tidak pernah belajar mengabaikannya**. Obatnya: buat sampel latih buatan yang bolong – salin beberapa sampel, lalu buang fitur yang diperkirakan akan hilang.

Kalahkan tolak banding: tiga hal yang perlu benar

Rekayasa fitur

Saring fitur yang tidak informatif (seleksi fitur), dan pakai pengetahuan Anda tentang persoalan untuk membuat fitur baru.

Prior arsitektur yang benar

Jaringan padat? ConvNet? Rekuren? Transformer? Atau – apakah deep learning memang pendekatan yang baik untuk tugas ini?

Konfigurasi pelatihan yang cukup baik

Fungsi rugi apa? Ukuran batch dan learning rate berapa?

TUGAS	AKTIVASI LAPIS AKHIR	FUNGSI RUGI	METRIK
Klasifikasi biner	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Multikelas, label tunggal	Softmax	Categorical crossentropy	Categorical accuracy, top-k, ROC AUC
Multikelas, multilabel	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Regresi	Tidak ada	Mean squared error	Mean absolute error

Mengapa metriknya tidak langsung dioptimalkan? Fungsi rugi harus bisa dihitung hanya dari satu mini-batch (idealnya bahkan dari satu titik data) dan harus **terdiferensialkan**. ROC AUC tidak memenuhi keduanya, jadi yang dioptimalkan adalah **pengganti**-nya, biasanya crossentropy – dengan harapan makin rendah crossentropy, makin tinggi ROC AUC.

Besarkan, lalu regularisasi

6.2.4 · Kembangkan model yang overfit

- ♦ Tambah lapis.
- ♦ Perbesar lapisnya.
- ♦ Latih lebih banyak epoch.

Model ideal berdiri **tepat di perbatasan** antara underfit dan overfit. Untuk tahu di mana perbatasannya, **Anda harus melewatinya lebih dulu.**

Peringatan yang sama, sekali lagi: tiap kali Anda memakai umpan balik validasi untuk menyetel model, informasi bocor ke dalam model. Beberapa kali tidak apa-apa; **dilakukan sistematis selama banyak iterasi, model akan overfit terhadap proses validasi itu sendiri** – walau tidak ada model yang dilatih langsung atas data validasi.

Sebagian besar pekerjaan ini bisa diotomasi dengan perangkat penyetelan hiperparameter seperti **KerasTuner** (bab 18). Dan bila unjuk kerja di himpunan uji ternyata jauh lebih buruk daripada di validasi: prosedur validasi Anda tidak andal, atau Anda sudah overfit ke data validasi. **Pindah ke protokol yang lebih andal, misalnya K-lipat berulang.**

6.2.5 · Regularisasi dan setel – tahap yang makan waktu paling banyak

- ♦ Coba arsitektur berbeda; tambah atau kurangi lapis.
- ♦ Tambahkan **dropout**.
- ♦ Kalau modelnya kecil, tambahkan **regularisasi L1 atau L2**.
- ♦ Coba hiperparameter lain – jumlah unit per lapis, learning rate.
- ♦ Bila perlu, ulangi **kurasi data atau rekayasa fitur**.

03

Menyerahkan model

Proyek tidak berakhir di notebook Colab yang bisa menyimpan model terlatih.

Menetapkan harapan – jangan bilang 'akurasi 98%'

- ♦ Harapan orang awam terhadap sistem AI sering **tidak realistis**: mereka mengira sistem *memahami* tugasnya dan punya akal sehat seperti manusia.
- ♦ Obatnya: **tunjukkan contoh cara model itu gagal** – terutama kesalahan klasifikasi yang terasa mengejutkan.
- ♦ Hindari pernyataan abstrak seperti "*modelnya berakurasi 98%*", yang **oleh kebanyakan orang dibulatkan ke 100%** dalam kepala mereka.

Bicaralah dalam **laju false-negative dan false-positive**, lalu terjemahkan ke angka harian yang bisa dibayangkan.

Dengan setelan ini, model deteksi fraud akan punya laju false-negative 5% dan false-positive 2,5%. Setiap hari, rata-rata 200 transaksi sah akan ditandai sebagai fraud dan dikirim ke pemeriksaan manual, dan rata-rata 14 transaksi fraud akan terlewat. Rata-rata 266 transaksi fraud akan tertangkap dengan benar.

Contoh penetapan harapan dari Chollet & Watson, bab 6.3.1

Bahas juga **pemilihan ambang** bersama pemangku kepentingan – ambang peluang yang berbeda menghasilkan laju false-negative dan false-positive yang berbeda. Keputusan itu menyangkut pertukaran yang **hanya bisa ditangani dengan pemahaman mendalam atas konteks bisnisnya**

Tiga cara menyerahkan model

REST API

Pakai bila: ada **akses internet andal**; **tidak ada kendala latensi ketat** (pulang-pergi sekitar **500 ms**); dan data masukannya **tidak sangat peka** – sebab data harus tersedia dalam bentuk terdekripsi di server.

PENCARI CITRA · PEREKOMENDASI · FRAUD · SATELIT

Di perangkat

Pakai bila: **latensi ketat** atau **konektivitas rendah**; model bisa dibuat cukup kecil; akurasi tertinggi **bukan** hal kritis; dan data masukannya **sangat peka** sehingga tak boleh didekripsi di server jauh.

SPAM TERENKRIPSI · KUE DI PABRIK

Di peramban

Pakai bila: ingin **memindahkan komputasi ke pengguna** (biaya server turun tajam); data harus tetap di perangkat pengguna; latensi ketat (hemat ~100 ms pulang-pergi jaringan); atau aplikasi harus tetap jalan **tanpa koneksi**.

VERSI WEB & DESKTOP APLIKASI OBROLAN

Peringatan untuk penyerahan di peramban: seluruh model **diunduh ke perangkat pengguna**. Pastikan tidak ada bagian model yang harus dirahasiakan – sebab dari model terlatih **biasanya masih mungkin memulihkan sebagian informasi tentang data pelatihannya**. Jangan publikasikan model yang dilatih atas data peka.

Mengekspor model: TensorFlow Serving dan ONNX

```
TensorFlow SavedModel PYTHON

model.export("path/to/location",
            format="tf_saved_model")

reloaded = tf.saved_model.load(
    "path/to/location")
predictions = reloaded.serve(input_data)
```

```
ONNX PYTHON

model.export("path/to/location",
            format="onnx")

ort_session = onnxruntime.InferenceSession(
    "path/to/location")
predictions = ort_session.run(None, input_data)
```

Keduanya bekerja dengan **mengangkat seluruh bobot model**

dan **graf komputasinya keluar dari program Python**, sehingga bisa dilayani dari banyak lingkungan berbeda – misalnya server C++. Kalau ini terdengar mirip mekanisme kompilasi di bab 3, **memang begitu**: TensorFlow Serving pada dasarnya pustaka untuk melayani graf `tf.function` dengan sehimpuan bobot tersimpan.

- **TensorFlow Lite** – inferensi di perangkat: Android, iOS, CPU ARM, Raspberry Pi, dan sebagian mikrokontroler. Formatnya sama dengan TensorFlow Serving. Runtime ONNX juga bisa jalan di perangkat bergerak.
- **TensorFlow.js** – menjalankan model di peramban; ia bahkan mengimplementasikan hampir seluruh API Keras (nama kerjanya dulu *WebKeras*). ONNX punya runtime JavaScript sendiri.
- Pilihan lain: **layanan awan terkelola** seperti Cloud AI Platform, yang mengurus pem-batch-an prediksi, penyeimbangan beban, dan penskalaan.

Mengoptimalkan model untuk inferensi

Pemangkasan bobot (weight pruning)

Tidak setiap koefisien pada tensor bobot menyumbang sama besar terhadap prediksi. Dengan hanya menyimpan yang paling berarti, jumlah parameter bisa **turun banyak** dengan ongkos kecil pada metrik. Seberapa banyak dipangkas = kendali Anda atas pertukaran ukuran lawan akurasi.

Kuantisasi bobot (weight quantization)

Model dilatih dengan bobot float32. Bobot itu bisa dikuantisasi ke **int8**, menghasilkan model khusus-inferensi yang **empat kali lebih kecil** tetapi tetap mendekati akurasi aslinya.

API kuantisasi bawaan Keras

PYTHON

```
model.quantize("int8") # tiap bobot dimampatkan jadi satu byte
```

Pengoptimalan ini **terutama penting** saat menyerahkan ke lingkungan dengan kendala daya dan memori ketat – ponsel dan perangkat tertanam – atau untuk aplikasi berlatensi rendah. Lakukan **sebelum** mengimpor ke TensorFlow.js atau mengeksport ke TensorFlow Lite.

Memantau dan merawat – pekerjaan yang tak berujung

- ① **Uji A/B teracak.** Sebagian kasus lewat model baru, sebagian lagi – kelompok kendali – tetap lewat proses lama. Setelah cukup banyak kasus, selisih hasilnya **bisa dikaitkan ke modelnya**, bukan ke perubahan lain.
- ② **Audit manual berkala** atas prediksi di data produksi. Infrastruktur anotasi yang sama bisa dipakai ulang: kirim sebagian data produksi untuk dianotasi manual, lalu bandingkan.
- ③ **Kalau audit manual mustahil**, cari jalur penilaian lain – misalnya survei pengguna, untuk sistem penandaan spam dan konten kasar.

Begitu model diluncurkan, Anda harus sudah bersiap melatih generasi berikutnya yang akan menggantikannya.

- ♦ Awasi **perubahan pada data produksi**. Apakah ada fitur baru? Apakah himpunan labelnya perlu diperluas atau diubah?
- ♦ Terus kumpulkan dan anotasi data, dan **perbaiki jalur anotasi** dari waktu ke waktu.
- ♦ Beri perhatian khusus pada **sampel yang tampak sulit diklasifikasikan** oleh model sekarang – sampel itulah yang **paling mungkin memperbaiki unjuk kerja**.

Yang wajib terbawa dari bab 6

- 1 **Tetapkan tugasnya dulu.** Pahami konteks, tujuan akhir, dan kendalanya; kumpulkan dan anotasi data; pilih cara mengukur keberhasilan.
- 2 Anda selalu membuat **dua hipotesis**. Sampai ada model yang bekerja, keduanya belum terbukti.
- 3 **Pilihan teknis juga pilihan etis.** Teknologi tidak pernah netral.
- 4 **Data harus mewakili produksi.** Waspada bias pencuplikan, kebocoran target, dan pergeseran konsep.
- 5 **Kalahkan tolok banding** → **besarkan sampai overfit** → **regularisasi dan setel**. Dalam urutan itu.
- 6 **Serahkan sesuai kendalanya:** REST API, di perangkat, atau di peramban – lalu pangkas dan kuantisasi untuk inferensi.
- 7 **Bicara dalam false-positive dan false-negative**, bukan persen akurasi.
- 8 **Tidak ada model yang bertahan selamanya.** Pantau, audit, dan siapkan generasi berikutnya sejak hari peluncuran.

NOTEBOOK

[03_ekspor_dan_kuantisasi.ipynb](#)

BAB BERIKUT

[Bab 7 — Menyelam ke Keras](#)